

IS - 3400 V3.0 RFID Reader

ISO 14443-A

ISO 14443-B

ISO 15693

Mifare Classic

Mifare UltraLight

Mifare Plus

Mifare NTAG

ICODE SLIX1, ICODE SLIX2

Encryption AES-128Bit, 3DES

날짜	버전	내용
2012.02.29	V1.0	V 1.0 Release
2012.10.20	V1.4	V 1.4 Release
2017.06.12	V3.0	V 3.0 Release
2017.10.02	V3.1	V 3.1 Release
2019.04.02	V3.2	V 3.2 Release

- 목 차 -

1. Specification

2. IS-3400 V3.0 구성

2.1 Connect 구분

2.2 USB Driver

4. SDK Test 탭

4.1 기본 SDK Funtion 사용법

5. SDK Funtion

(1) OpenSerialNumber

(2) OpenDescription

(3) Close

(4) GetDeviceNumber

(5) GetDescriptionExt

(6) GetSerialNumberExt

(7) GetTimeOut

(8) SetTimeOut

(9) GetCOMPort

(10) GetCOMPort_NoConnect

(11) WriteReadCommand

(12) WriteReadCommand 명령어 구조

(13) WriteCommand

(14) WriteCommand 명령어 구조

(15) ReadCommand

(16) ReadCommand 명령어 구조

(17) ReadExCommand

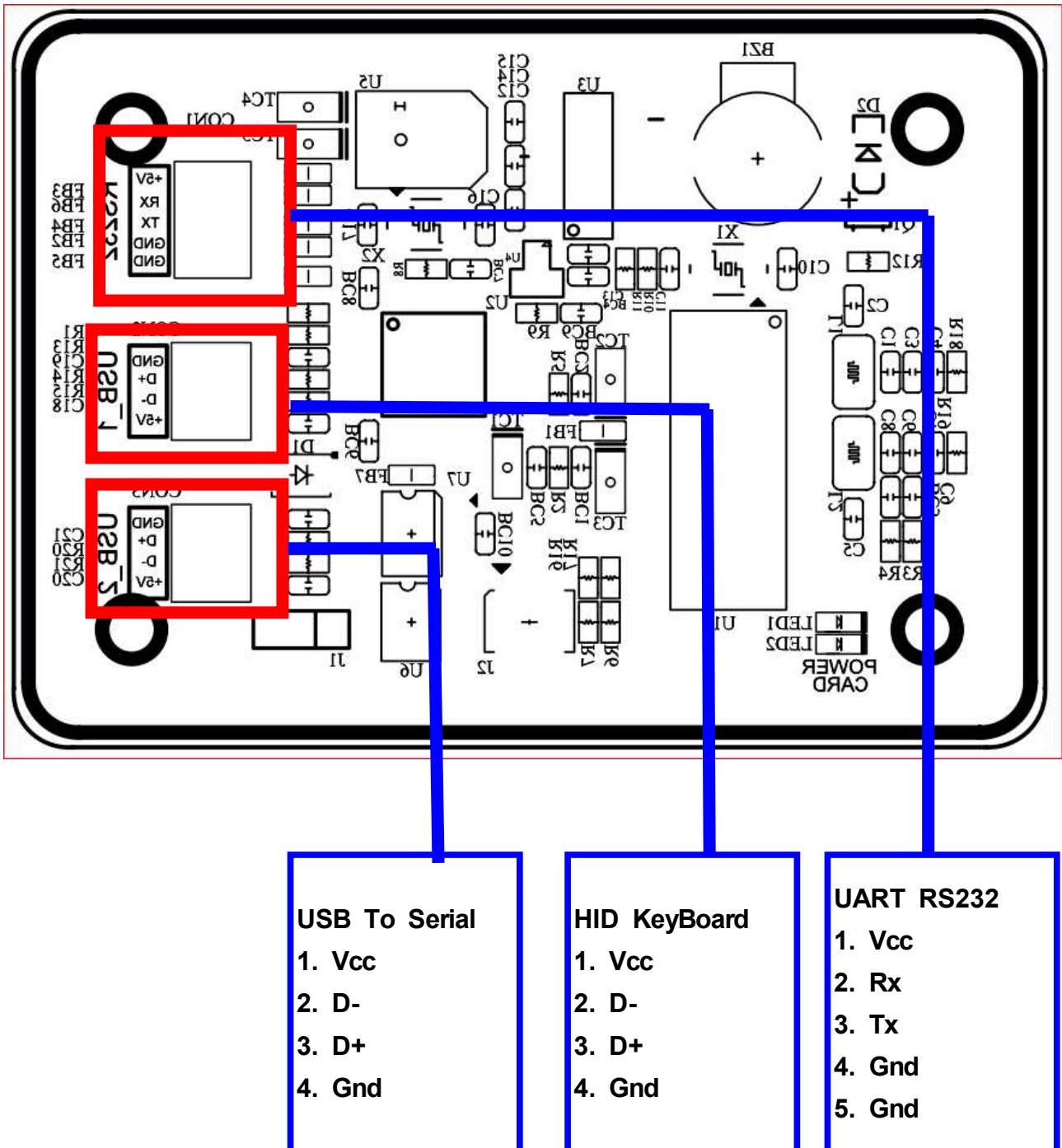
(18) ReadExCommand 명령어 구조

1. Specification

RF Frequency	13.56MHz
Power Supply	4.5 to 5.5V DC Operation
Supply Current	40mA @ 5V
Dimensions	70 x 50 x 6 mm
RF Protocol	ISO14443-A/B, ISO15693 Mifare Classic, Mifare UltraLight, Mifare Plus, Mifare NTAG, ICODE SLIX1, ICODE SLIX 2
Host Interface	RS232, TTL232, USB To Serial(FTDI USB Chip) USB HID Keyboard
Antennna	50-ohm Internal antenna
RF Power	150mW @ 5V
Read Range	50mm internal ant
Anticollision	Support(1tags)

2. IS-3400 V3.0 구성

2.1 Connect 구분



2.2 USB Driver

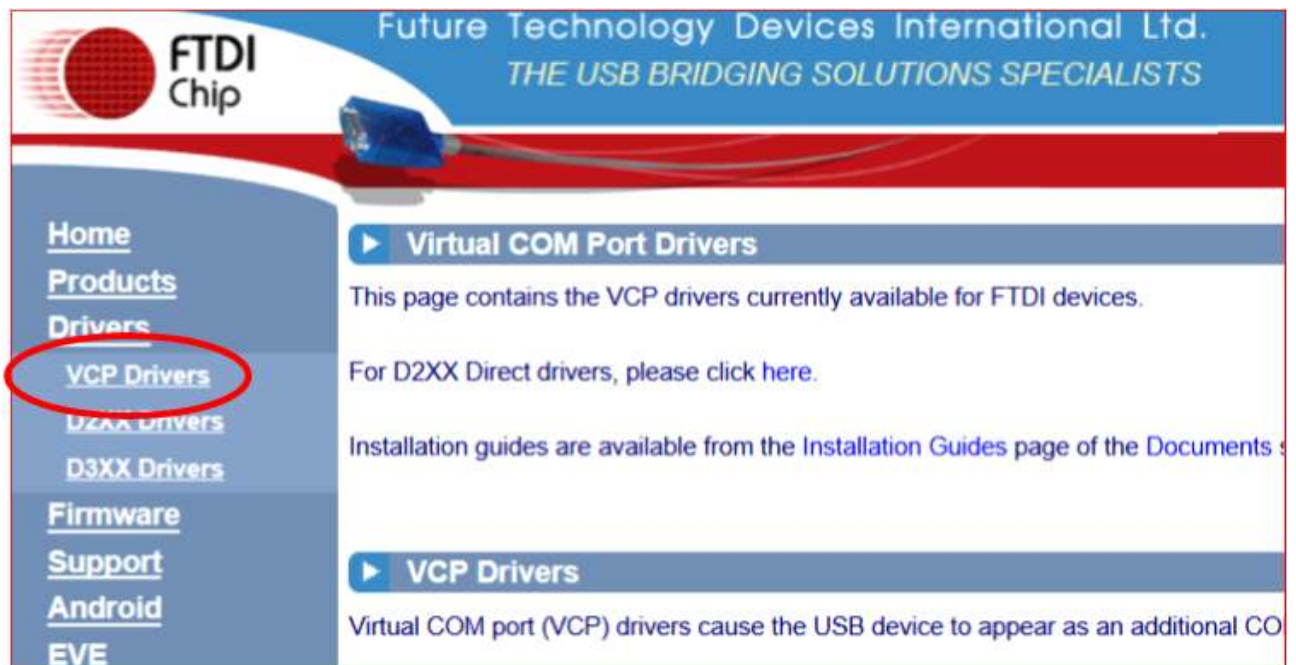
(1) HID USB KeyBoard

- Driver 설치가 필요 없이 자동으로 인식 됩니다.

(2) USB To Serial Driver

- USB Chip : FTDI230x
- 다운로드 사이트

<http://www.ftdichip.com/Drivers/VCP.htm>



4. SDK Test 탭

ISReadPro V3.1 Version

Menu Socket Connect

Serial Setup Exit

Common Crypt Calculator Crypt Command ISO14443ab ISO15693 ICODE SLIX 2 ICODE or EAS Mifare Classic Mifare UltraLight C Mifare NTag Mifare Plus SL3 ISO7816 SAM **SDK TEST** Auto F 4 ▶

FTDI USB Open / Close

FTDI USB State Serial Connect Serial Close

FTDI SerialNumber RFID01 115200 is_OpenSerialNumber

FTDI Description FT230X Basic UART 115200 is_OpenDescription

FTDI Close is_Close

SDK ETC Function

USB Number 0 FTDI SerialNumber RFID01 is_GetSerialNumber is_SetSerialNumber

USB Number 0 FTDI Description FT230X Basic UART is_GetDescription

FTDI USB State Close is_IsOpen

FTDI USB Device Read is_GetDeviceNumber

is_WriteReadCommand Function

CMD1 CMD1 Length Length Data

00 A3 00 00 is_WriteReadCommand

(Hex) (Hex) (Hex) (Hex) (Hex) ☒ BuzzerOn

00 08 147D72A0500104E0 State (return value) IS_OK

PROTOCOL

Send Data Format (Hex)	STX	CMD1	CMD2	Length	Length	Data	Checksum	ETX
Receive Data Format (Hex)	STX	CMD1	CMD2	State	Length	Length	Data	Checksum ETX

Tx/Rx Protocol

SDK is_WriteReadCommand(00, A3, 0, Data, readLength = 8, ref Byte[]) == IS_OK

SDK is_WriteReadCommand(00, 23, 0, Data, readLength = 8, ref Byte[]) == IS_OK

SDK is_OpenSerialNumber(RFID01, 115200) == IS_OK

SDK is_GetDescription(0, ref string value) == IS_OK

SDK is_GetSerialNumber() == IS_OK

SDK is_WriteReadCommand(00, 23, 0, Data, readLength = 8, ref Byte[]) == IS_OK

RF Power State ● TCP Socket ● Serial Connect ●

4.1 기본 SDK Function 사용법

① Serial 통신이 ISReaderPro 3.0으로 연결 되어 있으면 SDK로 연결이 안됩니다. Serial 연결을 Close 하고 테스트 하시면 됩니다.

② is_OpenSerialNumber 함수를 테스트 합니다. FTDI SerialNumber를 ⑤에서 확인 후 SerialNumber와 통신 속도를 입력 후 연결을 시도 하세요.

③ is_OpenDescription 함수를 테스트 합니다. FTDI Description를 ⑦에서 확인 후 통신 속도를 입력 후 연결을 시도 하세요.

④ is_Close 함수 테스트, 통신 연결을 Close 합니다.

⑤ is_GetSerialNumber 함수 테스트, FTDI SerialNumber를 확인 할수 있습니다. 통신 연결이 되어 있으면 읽어 올수 없습니다. USB Number는 FTDI USB To Serial 이 연결 되어 있는 번호를 나타 냅니다. 현재 FTDI 사의 USB To Serial 칩이 몇 개 연결 되어 있는지는 ⑨ 통해 확인 할수 있습니다.
예) 2개가 연결 되어 있으면 USB Number는 0, 1입니다.

⑦ is_GetDescription 함수를 테스트, FTDI Description을 확인 할수 있습니다.

⑧ is_IsOpen 현재 연결 상태를 확인 할수 있습니다.

⑨ is_GetDeviceNumber 함수를 테스트, 현재 FTDI USB to Serial 칩으로 연결 되어 있는 개수를 읽어 옵니다.

Ⓐ is_WriteReadCommand 함수 테스트, 인자에 시리얼 통신에서 사용한 값과 동일하게 입력 하고 사용 가능합니다. 명령 실행을 요청 하고, 응답 할때 까지 대기 하여 응답 데이터를 읽어 옵니다. 응답 시간을 짧게 사용 해야 한다면 is_WriteCommand 명령어 사용 후 스레드로 is_ReadExCommand 함수를 이용 하여 읽으시면 됩니다.

5. SDK Funtion

(1) OpenSerialNumber

FTDI SerialNumber로 USB 포트를 오픈 연결 합니다.

```
IS_STATUS OpenSerialNumber
(
    ref ftHandle,
    byte[] SerialNumberReadBuffer,
    long BaudRate
)
```

Arguments :

ftHandle: IntPtr 타입 변수 이며, 포트 오픈 되면 핸들을 받아 옵니다.

SerialNumberReadBuffer : 연결할 FTDI SerialNumber

BaudRate: 연결할 BaudRate

BaudRate
4800,
9600
19200
38400
57600
115200
230400

Return Value

성공시 IS_OK = 0을 리턴 합니다.

Example

```
IntPtr ftHandle = IntPtr.Zero;
byte[] SerialNumber = Encoding.UTF8.GetBytes("RFID01");
// USB To Serial 연결합니다. 통신속도는115200baud 입니다.
if (mDevice.OpenSerialNumber(ref ftHandle, SerialNumber, 115200)
    == isD2xx.IS_STATUS.IS_OK)
{
    System.Console.WriteLine(" USB To Serial와 통신연결 완료");
}
```


(2) OpenDescription

FTDI Description으로 USB 포트를 오픈 연결 합니다.

```
IS_STATUS OpenDescription
(
    ref ftHandle,
    byte[] DescriptionReadBuffer,
    long BaudRate
)
```

Arguments :

ftHandle : IntPtr 타입 변수이며, 포트 오픈 되면 핸들 값 저장

DescriptionReadBuffer, : 연결할 FTDI Description

Description 데이터는 "FT230X Basic UART" 입니다.

BaudRate : 연결할 BaudRate

BaudRate
4800,
9600
19200
38400
57600
115200
230400

Return Value

성공시 IS_OK = 0을 리턴 합니다.

Example

```
IntPtr ftHandle = IntPtr.Zero;
byte[] descriptNumber = Encoding.UTF8.GetBytes("FT230X Basic UART");
// USB To Serial 연결합니다. 통신속도는115200baud 입니다.
if (mDevice.OpenDescription(ref usbHandle, descriptNumber, baud)
    == isD2xx.IS_STATUS.IS_OK)
{
    System.Console.WriteLine(" USB To Serial와 통신연결 완료");
}
```

(3) Close

FTDI SerialNumber로 USB 포트를 오픈 연결 합니다.

```
IS_STATUS Close  
(  
    IntPtr ftHandle  
)
```

Arguments :

ftHandle : 핸들 변수

Return Value

성공시 IS_OK = 0을 리턴 합니다.

Example

```
if (mDevice.Close(ftHandle))  
    == isD2xx.IS_STATUS.IS_OK)  
{  
    System.Console.WriteLine(" USB To Serial와 통신 종료 완료");  
}
```

(4) GetDeviceNumber

FTDI USB To Serial 포트가 몇 개 인지 읽어 옵니다.

```
IS_STATUS GetDeviceNumber  
(  
    ref short deviceNumber  
)
```

Arguments :

deviceNumber : FTDI USB To Serial 연결된 개수 받아 옵니다.

Return Value

성공시 IS_OK = 0을 리턴 합니다.

Example

```
short deviceNumber = 0;  
if (mDevice.GetDeviceNumber(ref deviceNumber)  
    == isD2xx.IS_STATUS.IS_OK)  
{  
    System.Console.WriteLine("FTDI USB To Serial 연결된개수: " + deviceNumber);  
}
```

(5) GetDescriptionExt

FTDI USB To Serial Description 값을 읽어 옵니다.

```
IS_STATUS GetDeviceNumber  
(  
    long deviceNumber,  
    byte[] DescriptionReadBuffer,  
    ref short DescriptionLength  
)
```

Arguments :

DeviceNumber : FTDI USB To Serial 연결되어 있는 번호 0, 1, 2, 형태로 됩니다.

첫 번째 있는 FTDI USB To Serial 은 0 이 됩니다.

DescriptionReadBuffer : IS-3400 V3.0 버전은 “FT230X Basic UART” 입니다.

DescriptionLength : 저장할 데이터 바이트 사이즈

Return Value

성공시 IS_OK = 0을 리턴 합니다.

Example

```
byte[] decsriptNumber = new byte[64];  
short length = 0;  
string str;  
if (mDevice.is_GetDescription(0, decsriptNumber, ref length)  
    == isD2xx.IS_STATUS.IS_OK)  
{  
    str = System.Text.Encoding.UTF8.GetString(decsriptNumber);  
}
```

(6) GetSerialNumberExt

FTDI USB To Serial SerialNumber 값을 읽어 옵니다.

```
IS_STATUS GetSerialNumberExt
(
    long deviceNumber,
    byte[] SerialNumberReadBuffer,
    ref short SerialLength
)
```

Arguments :

DeviceNumber : FTDI USB To Serial 연결되어 있는 번호 0, 1, 2, 형태로 됩니다.
첫 번째 있는 FTDI USB To Serial 은 0 이 됩니다.

SerialNumberReadBuffer: FTDI USB to Serial 값 저장할 버퍼

SerialLength : 저장할 데이터 바이트 사이즈

Return Value

성공시 IS_OK = 0을 리턴 합니다.

Example

```
string str = string.Empty;
byte[] serialNumber = new byte[64];
short length = 0;
if (mDevice.is_GetSerialNumber(0, serialNumber, ref length )
    == isD2xx.IS_STATUS.IS_OK))
{
    str = System.Text.Encoding.UTF8.GetString(serialNumber);
}
```

(7) GetTimeOut

FTDI USB To Serial Open완료 후 타임아웃 설정시간을 확인 할 수 있습니다.

```
IS_STATUS GetTimeOut
(
    IntPtr ftHandle,
    ref UInt32 readTimeOut_milliseconds,
    ref UInt32 writeTimeOut_milliseconds
)
```

Arguments :

ftHandle : 연결된 핸들 변수
readTimeOut_milliseconds : 응답대기 최대 시간
writeTimeOut_milliseconds : 전송대기 최대 시간

Return Value

성공시 IS_OK = 0을 리턴 합니다.

Example

```
UInt32 readTimeout = 0, writeTimeout = 0;
if (mDevice.GetTimeOut(ftHandle, ref readTimeout, ref writeTimeout) ==
    isD2xx.IS_STATUS.IS_OK)
{
    System.Console.WriteLine(" Read Timeout : " + readTimeout);
    System.Console.WriteLine(" Write Timeout : " + writeTimeout);
}
```

(8) SetTimeOut

FTDI USB To Serial Open완료 후 타임아웃 설정시간을 설정 할 수 있습니다.

```
IS_STATUS SetTimeOut
(
    IntPtr ftHandle,
    UInt32 readTimeOut_milliseconds,
    UInt32 writeTimeOut_milliseconds
)
```

Arguments :

ftHandle : 연결된 핸들 변수

readTimeOut_milliseconds : 응답대기 최대 시간

writeTimeOut_milliseconds : 전송대기 최대 시간

Return Value

성공시 IS_OK = 0을 리턴 합니다.

참조 : is_OpenSerialNumber, is_OpenDescription 함수로 연결 후 사용 가능 합니다.
is_WriteCommand, is_WriteReadCommand, is_ReadCommand, is_ReadExCommand
응답 대기 시간, 전송대기 시간을 설정 할수 있습니다.
기본 값 500ms로 설정 되어 있습니다.

Example

```
UInt32 readTimeout = 500, writeTimeout = 500;

if (mDevice.iSetTimeOut(ftHandle, readTimeout, writeTimeout)
    == isD2xx.IS_STATUS.IS_OK)
{
    System.Console.WriteLine(" 설정완료");
}
```

(9) GetCOMPort

FTDI USB To Serial Open 후 사용 가능 하며, 현재 Open한 ComPort를 읽어 옵니다.

```
IS_STATUS GetCOMPort
(
    IntPtr ftHandle,
    ref long comPor
)
```

Arguments :

ftHandle : 연결된 핸들 변수
comPor : ComPort Number를 저장할 변수

Return Value

성공시 IS_OK = 0을 리턴 합니다.

Example

```
long buad = 0;
if (mDevice.GetCOMPort(ftHandle, ref buad) == isD2xx.IS_STATUS.IS_OK)
    Console.WriteLine("ComPort Number : {0}", buad);
```


(10) GetCOMPort_NoConnect

FTDI USB To Serial Open 하지 않은 상태에서 ComPort를 읽어 옵니다.

```
IS_STATUS GetCOMPort_NoConnect  
(  
    long DeviceNumber,  
    ref long comPor  
)
```

Arguments :

DeviceNumber: FTDI USB To Serial 연결되어 있는 번호 0, 1, 2, 형태로 됩니다.

첫 번째 있는 FTDI USB To Serial 은 0 이 됩니다.

comPor : ComPort Number를 저장할 변수

Return Value

성공시 IS_OK = 0을 리턴 합니다.

Example

```
long buad = 0;  
if (mDevice.GetCOMPort_NoConnect(0, ref buad)  
    == isD2xx.IS_STATUS.IS_OK)  
    Console.WriteLine("ComPort Number : {0}", buad);
```

(11) WriteReadCommand

RFID 리더기의 명령어를 전송 하고 데이터를 읽어 오는 함수입니다.

```
IS_STATUS WriteReadCommand
(
    IntPtr ftHandle,
    byte cmd1,
    byte cmd2,
    UInt16 writeLength,
    byte[] writeData,
    ref UInt16 length,
    byte[] readData
)
```

Arguments :

ftHandle : 연결된 핸들 변수
 cmd1 : 상위 명령어
 cmd2 : 하위 명령어
 writeLength : 전송 명령어 데이터 길이
 writeData : 전송 명령어 데이터
 readlength : 응답 받은 데이터 길이
 readData : 응답 받은 데이터 버퍼

Return Value

성공시 IS_OK = 0을 리턴 합니다.

Example

```
byte[] writeBuffer = new byte[1024];
byte[] readBuffer = new byte[1024];
ushort readLength = 0, writeLength = 0;
if (mDevice.WriteReadCommand(ftHandle, CMD1_COMMON, CMD2_COMMON_VERSION,
    writeLength, writeData, ref readLength, ref readData)
    == isD2xx.IS_STATUS.IS_OK)
{
    byte[] mversion = new byte[readLength];
    for (int i = 0; i < readLength; i++)
        mversion[i] = readData[i];

    System.Console.WriteLine(Encoding.Default.GetString(mversion));
}
```


(13) WriteCommand

RFID 리더기의 명령어를 전송 하는 함수입니다.

```
IS_STATUS WriteCommand
(
    IntPtr ftHandle,
    byte cmd1,
    byte cmd2,
    UInt16 writeLength,
    byte[] writeData
)
```

Arguments :

ftHandle : 연결된 핸들 변수
 cmd1 : 상위 명령어
 cmd2 : 하위 명령어
 writeLength : 전송 명령어 데이터 길이
 writeData : 전송 명령어 데이터

Return Value

성공시 IS_OK = 0을 리턴 합니다.

Example

```
byte[] writeBuffer = new byte[1024];
ushort writeLength = 0;
if (mDevice.WriteCommand(ftHandle, CMD1_COMMON, CMD2_COMMON_VERSION,
    writeLength, writeData) == isD2xx.IS_STATUS.IS_OK)
{
    System.Console.WriteLine("전송 완료");
}
```

(14) WriteCommand 명령어 구조

The screenshot shows the ISO15693 Single Block Write configuration. The 'Single Block Write' button is highlighted. Below the configuration, the 'PROTOCOL' section shows the data format for the command. The fields are: CMD1 (03), CMD2 (A3), Length (00), and Data (0012345678). Arrows point from these fields to the corresponding parameters in the WriteCommand function signature below.

is_WriteCommand(0x03, 0x23 + 0x80, writeLength, writeData)

(15) ReadCommand

RFID 리더기의 데이터를 읽어 오는 함수입니다.

```
IS_STATUS ReadCommand
(
    IntPtr ftHandle,
    ref UInt16 length,
    byte[] readData
)
```

Arguments :

ftHandle : 연결된 핸들 변수
 readlength : 응답 받은 데이터 길이
 readData : 응답 받은 데이터 버퍼

Return Value

성공시 IS_OK = 0을 리턴 합니다.

Example

```
byte[] writeBuffer = new byte[1024];
byte[] readBuffer = new byte[1024];
ushort readLength = 0, writeLength = 0;
if (mDevice.WriteCommand(ftHandle, CMD1_COMMON, CMD2_COMMON_VERSION,
    writeLength, writeData) == isD2xx.IS_STATUS.IS_OK)
{
    if (mDevice.ReadCommand(ftHandle, ref readLength, ref readData)
        == isD2xx.IS_STATUS.IS_OK)
    {
        byte[] mversion = new byte[readLength];
        for (int i = 0; i < readLength; i++)
            mversion[i] = readData[i];

        System.Console.WriteLine(Encoding.Default.GetString(mversion));
    }
}
```

(16) ReadCommand 명령어 구조

The screenshot displays the ISO15693 command configuration interface. The 'ISO15693 Single Block Read / Write / Lock' section is active, showing 'Block Number' as 0 and 'Block Data' as 12345678. The 'Single Block Read' button is highlighted. Below this, the 'ISO15693 Multiple Block Read / Write' section shows 'Block Number' as 0 and 'Block Count' as 10. The 'Block Data' field contains a long hexadecimal string. The 'ISO15693 Get SystemInformation' section shows 'Data' as 0F064AB939500104E000001B0301. The 'ISO15693 ETC Command' section shows 'ISO 15693 StayOutlet', 'ISO 15693 Select', and 'ISO 15693 Reset To Ready' buttons. The 'PROTOCOL' section at the bottom shows the data format for sending and receiving commands. The 'Send Data Format (Hex)' table has columns for STX (01), CMD1 (03), CMD2 (A1), Length (00), Length (01), Data (00), CheckSum (A5), and ETX (03). The 'Receive Data Format (Hex)' table has columns for STX (01), CMD1 (03), CMD2 (21), State (01), Length (00), Length (04), Data (12345678), CheckSum (3D), and ETX (03). Arrows point from the 'Length' and 'Data' fields in the 'Receive Data Format' table to the 'ref readLength' and 'ref readData' parameters in the code snippet below.

PROTOCOL								
Send Data Format (Hex)	STX	CMD1	CMD2	Length	Length	Data	CheckSum	ETX
	01	03	A1	00	01	00	A5	03
Receive Data Format (Hex)	STX	CMD1	CMD2	State	Length	Length	Data	CheckSum
	01	03	21	01	00	04	12345678	3D

mDevice.is_ReadCommand(ref readLength, ref readData)

(17) ReadExCommand

RFID 리더기의 명령어와 데이터를 읽어 오는 함수입니다.

* 읽기 전용으로 쓰레드에서 동작 할 때 사용하면 편리 합니다.

```
IS_STATUS ReadExCommand
(
    IntPtr ftHandle,
    ref byte cmd1,
    ref byte cmd2,
    ref UInt16 length,
    byte[] readData
)
```

Arguments :

ftHandle : 연결된 핸들 변수
 cmd1 : 상위 명령어 응답 받을 Byte 변수
 cmd2 : 하위 명령어 응답 받을 Byte 변수
 readlength : 응답 받은 데이터 길이
 readData : 응답 받은 데이터 버퍼

Return Value

성공시 IS_OK = 0을 리턴 합니다.

Example

```
byte[] writeBuffer = new byte[1024];
byte[] readBuffer = new byte[1024];
ushort readLength = 0, writeLength = 0;
byte cmd1 = 0, cmd2 = 0
if (mDevice.WriteCommand(ftHandle, CMD1_COMMON, CMD2_COMMON_VERSION,
    writeLength, writeData) == isD2xx.IS_STATUS.IS_OK)
{
    if (mDevice.ReadExCommand(ftHandle, ref cmd1, ref cmd2,
        ref readLength, ref readData) == isD2xx.IS_STATUS.IS_OK)
    {
        byte[] mversion = new byte[readLength];
        for (int i = 0; i < readLength; i++)
            mversion[i] = readData[i];

        System.Console.WriteLine(Encoding.Default.GetString(mversion));
    }
}
```


(18) ReadExCommand 명령어 구조

The screenshot shows the ISO15693 configuration interface. In the 'ISO15693 Single Block Read / Write / Lock' section, 'Block Number' is 0, 'Block Data' is 12345678, and 'BuzzerOn' is checked. The 'Single Block Read' button is highlighted. In the 'PROTOCOL' section, the 'Send Data Format (Hex)' table shows STX: 01, CMD1: 03, CMD2: A1, Length: 00, Length: 01, Data: 00, CheckSum: A5, ETX: 03. The 'Receive Data Format (Hex)' table shows STX: 01, CMD1: 03, CMD2: 21, State: 01, Length: 00, Length: 04, Data: 12345678, CheckSum: 30, ETX: 03. Arrows indicate that the code arguments correspond to these values: cmd1 (03), cmd2 (21), readLength (04), and readData (12345678).

```
if (mDevice.is_ReadExCommand(ref cmd1, ref cmd2, ref readLength, ref readData) != IS_D2XX.NET.IS_D2XX.IS_STATUS.IS_OK)
```